



API Performance Studio

User Manual

Version 0.2.0

Fire. Measure. Validate.

Table of Contents

1. Introduction
2. Installation
3. App Overview
4. Sidebar — Environments & Microservices
5. Feature Groups — Organizing Tests
6. Creating & Editing Tests
7. Authentication Setup
8. Settings
9. Running Performance Tests
10. Analyzing Results
11. Export & Import
12. Desktop vs Web Comparison
13. Tips & Best Practices
14. Quick Reference

1. Introduction

RedfireForge is a desktop and web application for API performance testing. It lets you define HTTP tests visually, execute them with configurable concurrency, validate responses against expected schemas, and analyze the results — all without writing a single line of test script.

RedfireForge is available as a native desktop application (macOS, Windows, Linux) and as a web application running in your browser.

Key Capabilities

- Visual HTTP test builder — no scripting required
- Hierarchical organization: Feature Groups → Scenarios → Tests
- OAuth2, Bearer Token, Basic Auth, and API Key authentication
- Global auth profiles with 4-tier inheritance
- Sequential, Parallel, and Ramp-up execution modes
- Real-time execution progress and result metrics
- Export/Import with intelligent conflict resolution
- Drag-and-drop test organization
- Cross-platform: macOS, Windows, Linux, and Web

2. Installation

Desktop Application (Recommended)

Platform	Installer	How to Install
macOS	.dmg file	Open DMG → drag RedfireForge to Applications
Windows	.msi installer	Double-click → follow the setup wizard
Linux	.AppImage / .deb	Make AppImage executable and run, or install .deb

macOS first launch: Right-click the app → Open → click "Open" in the dialog to bypass Gatekeeper security.

Web Application

If using the web version, you need Node.js installed. Run the following commands:

`npm install` — install dependencies

`npm run dev` — start the dev server

Open `http://localhost:5173` in your browser.

3. App Overview

When you launch RedfireForge, you'll see the main interface with three primary areas: the header with navigation tabs, the sidebar for filtering, and the main content area.

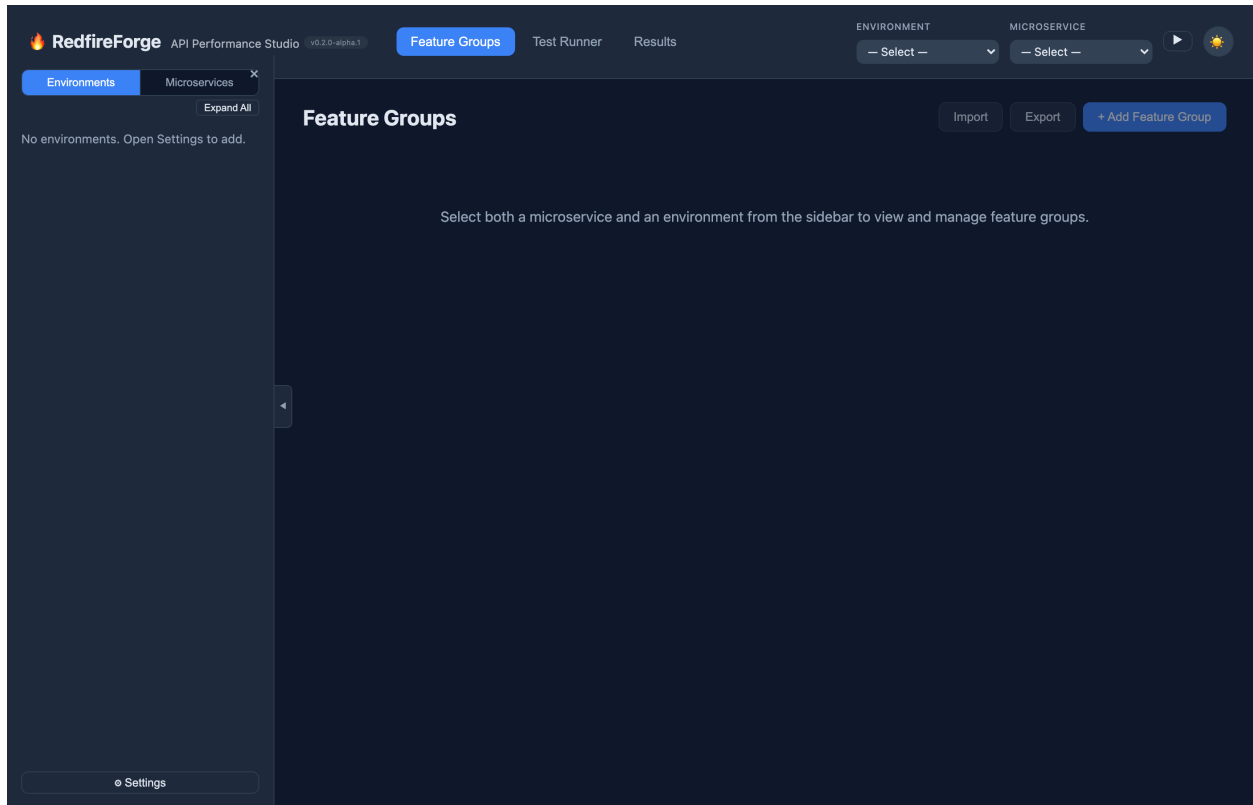


Figure 1: RedfireForge main interface — Feature Groups view

The interface has three main navigation tabs:

Tab	Purpose
Feature Groups	Create, organize, and edit your API tests
Test Runner	Configure and execute performance test runs
Results	View historical results, metrics, and charts

4. Sidebar — Environments & Microservices

The left sidebar lets you filter Feature Groups by Environment and Microservice. This is how you switch between dev, test, staging, and production configurations.

Step 1: Select an Environment

Click on an environment (e.g., d01, t01, p01) in the sidebar to filter Feature Groups that belong to that environment.

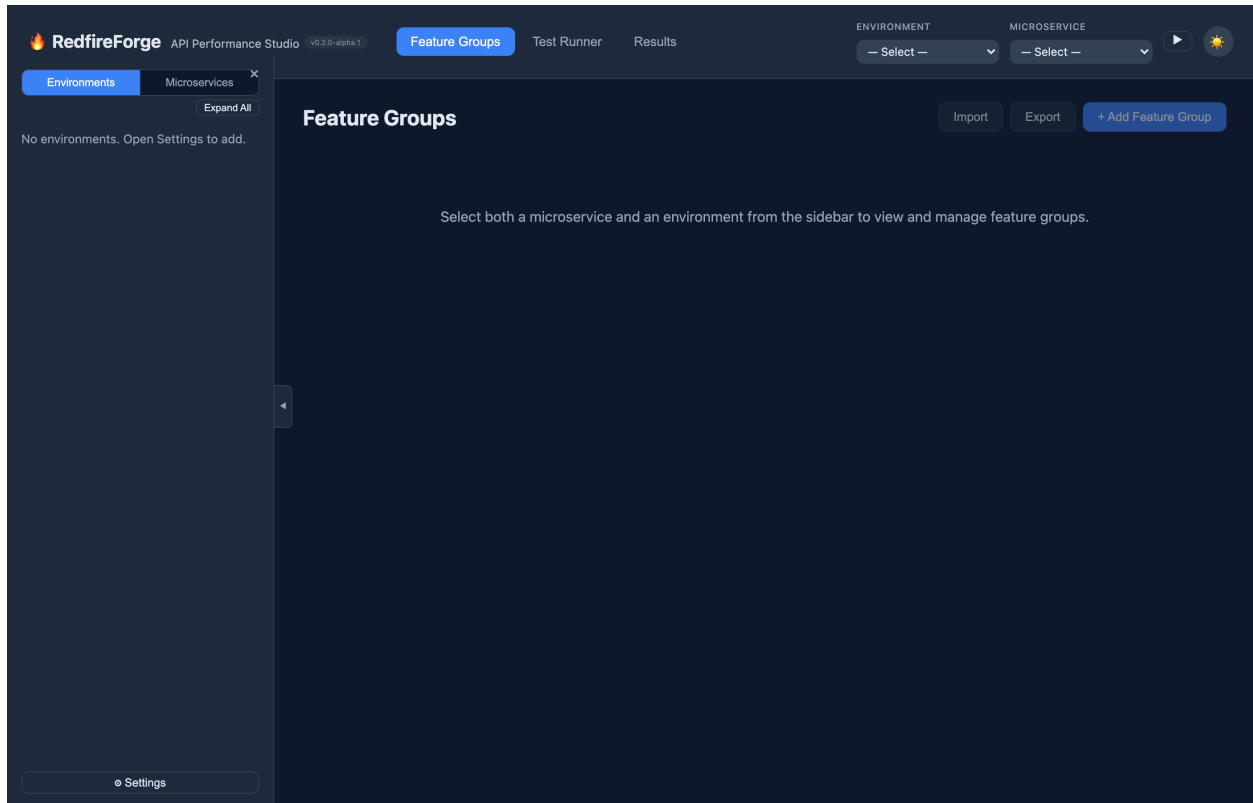


Figure 2: Sidebar with an environment selected

Step 2: Switch to Microservices

Click the 'Microservices' tab in the sidebar to filter by microservice instead.

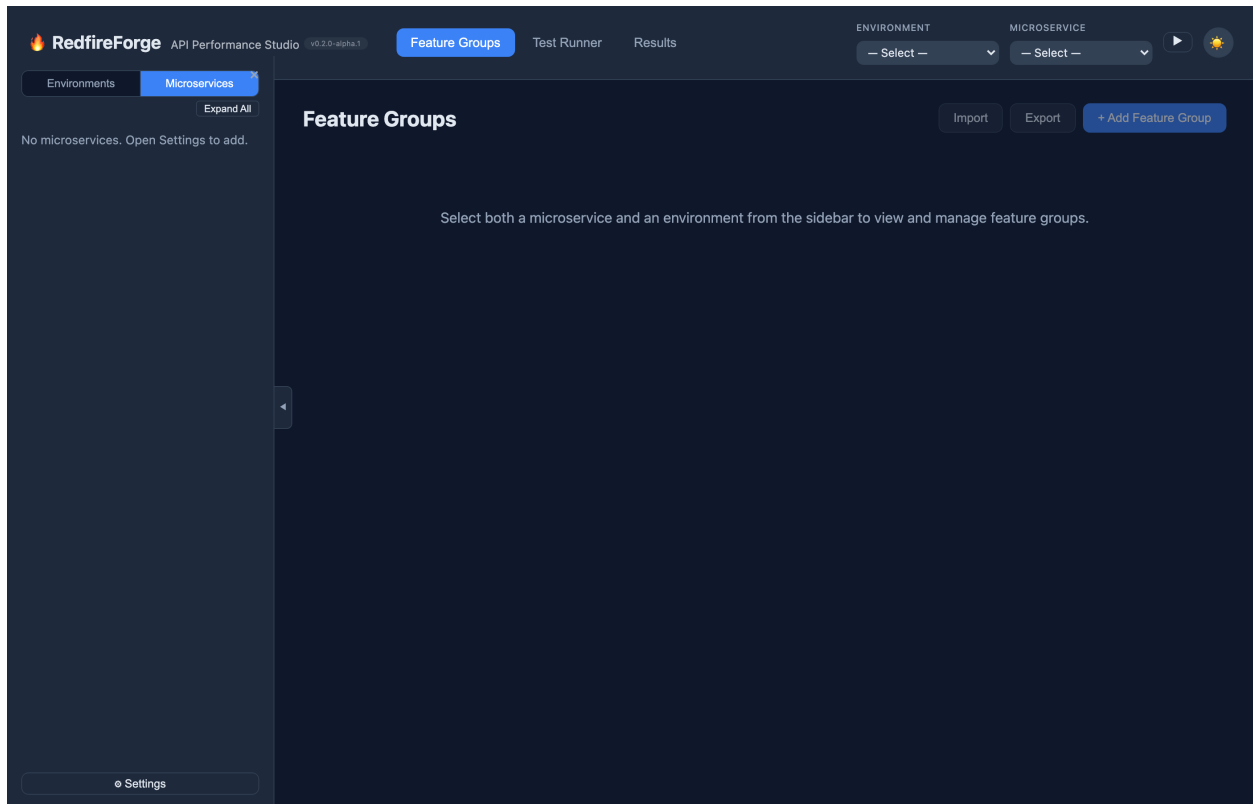


Figure 3: Sidebar showing the Microservices tab

5. Feature Groups — Organizing Tests

Feature Groups are the top-level containers for organizing your tests. Each Feature Group is tied to an Environment and Microservice, and contains one or more Scenarios.

Test Hierarchy

Level	Description	Example
Feature Group	Top-level container	"Onboarding API Tests"
Scenario	A test flow within a group	"Happy Path — New User"
Test	A single HTTP request + assertions	"POST /api/users — 201"

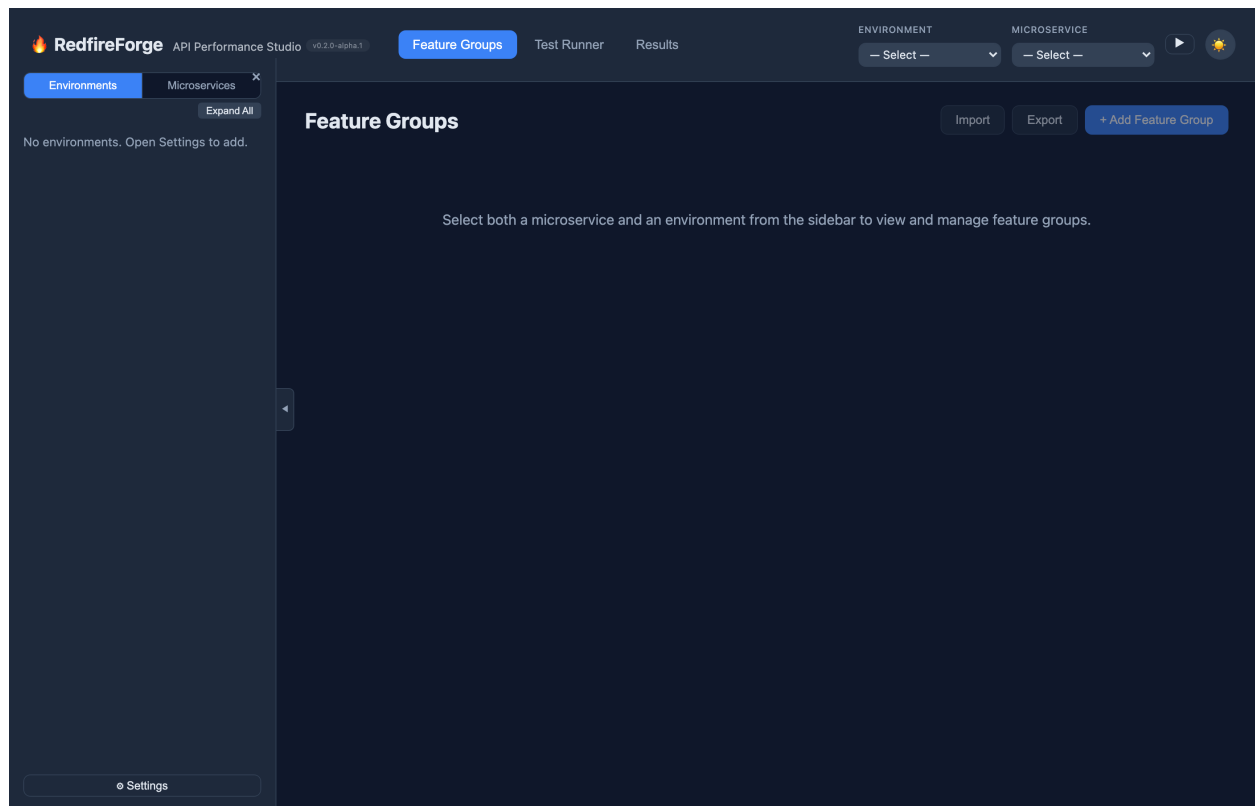


Figure 4: Feature Groups with scenarios and tests expanded

Use the **Expand All** button to see all scenarios and tests at once. You can drag-and-drop scenarios and tests to reorder or move them between groups.

6. Creating & Editing Tests

Step 1: Add a Feature Group

Click '+ Add Feature Group' in the Feature Groups tab. Give it a name, select the Environment and Microservice.

Step 2: Add a Scenario

Inside a Feature Group, click '+ Add Scenario'. Name it to describe the test flow (e.g., 'Login Flow').

Step 3: Add a Test

Inside a Scenario, click '+ Add Test'. Configure the HTTP request:

Test Configuration Fields

Field	Description
Name	A descriptive name for the test
Method	HTTP method: GET, POST, PUT, DELETE, PATCH
URL	The full request URL
Headers	Key-value pairs for request headers
Body	Request body (for POST/PUT/PATCH)
Expected Status	The expected HTTP status code (e.g., 200, 201)
Validation	JSON response validation rules
Auth	Override authentication for this specific test

7. Authentication Setup

RedfireForge supports a 4-tier authentication inheritance model. Configure auth once at the top level and let it cascade down:

Level	Inherits From	Override?
Global Auth Profile	—	Base configuration
Feature Group	Global Profile	Can override
Scenario	Feature Group	Can override
Test	Scenario	Can override

Supported Auth Types

- **OAuth2 (Client Credentials)** — Token URL, Client ID, Client Secret, Scope
- **Bearer Token** — Static token value
- **Basic Auth** — Username and password
- **API Key** — Key name, value, sent in header or query

Use the "Verify" button to test your credentials before running tests.

8. Settings

Access Settings by clicking the ■ Settings button in the sidebar. Here you can manage Environments, Microservices, Global Auth Profiles, and access Export/Import functionality.

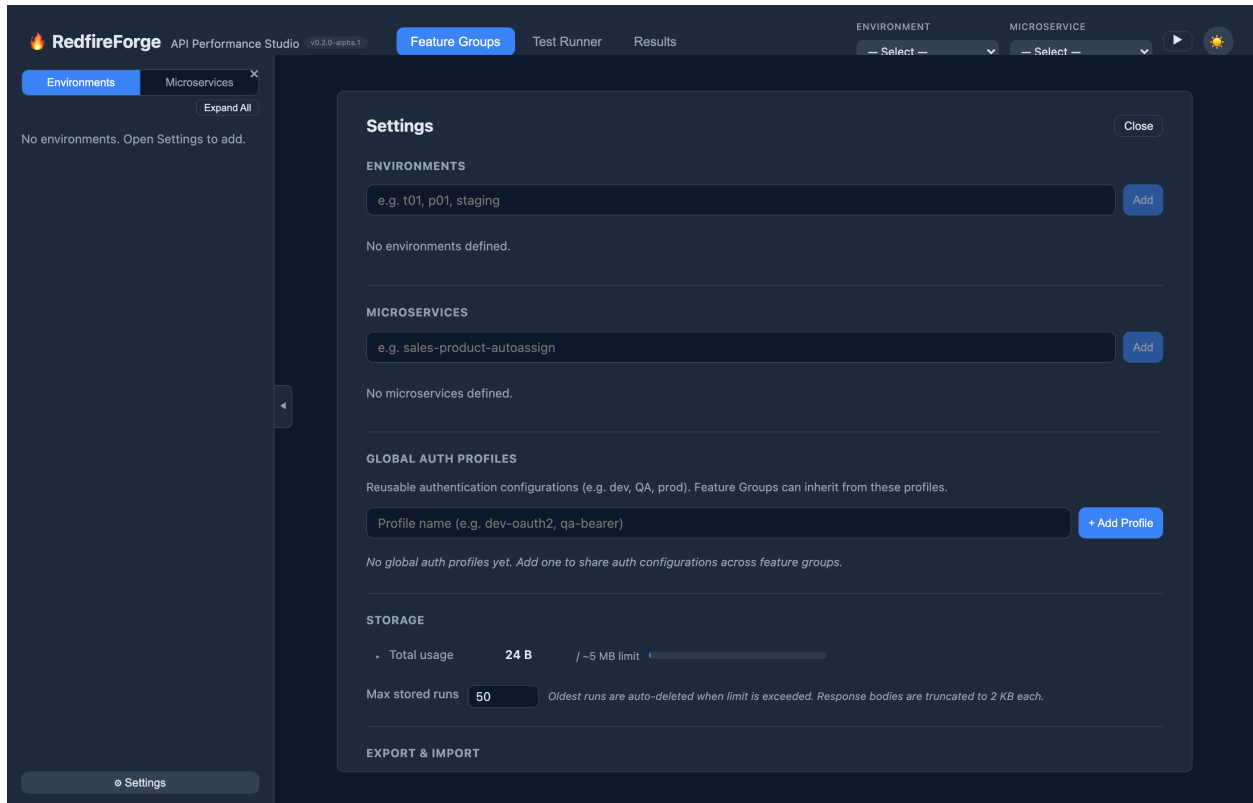


Figure 5: Settings modal — Environments and Microservices

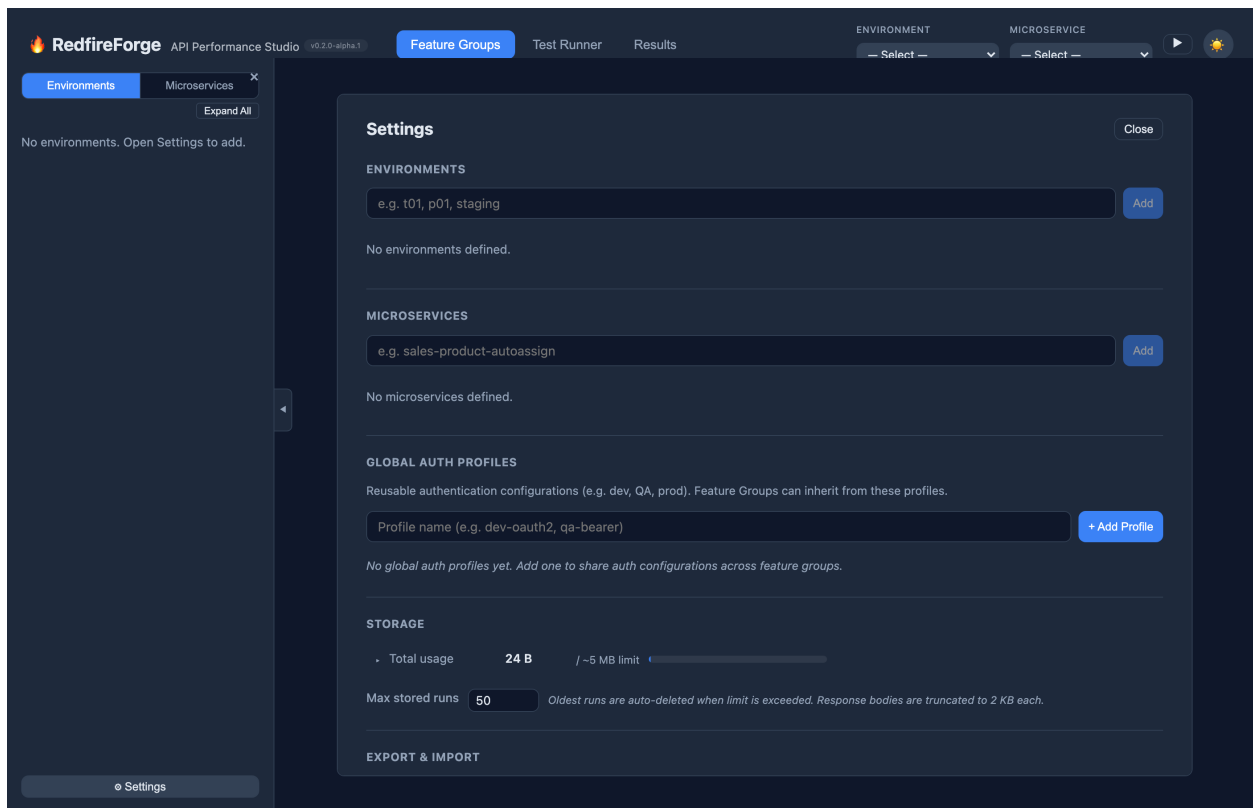


Figure 6: Settings — Global Authentication Profiles

9. Running Performance Tests

Switch to the Test Runner tab to configure and execute performance tests.

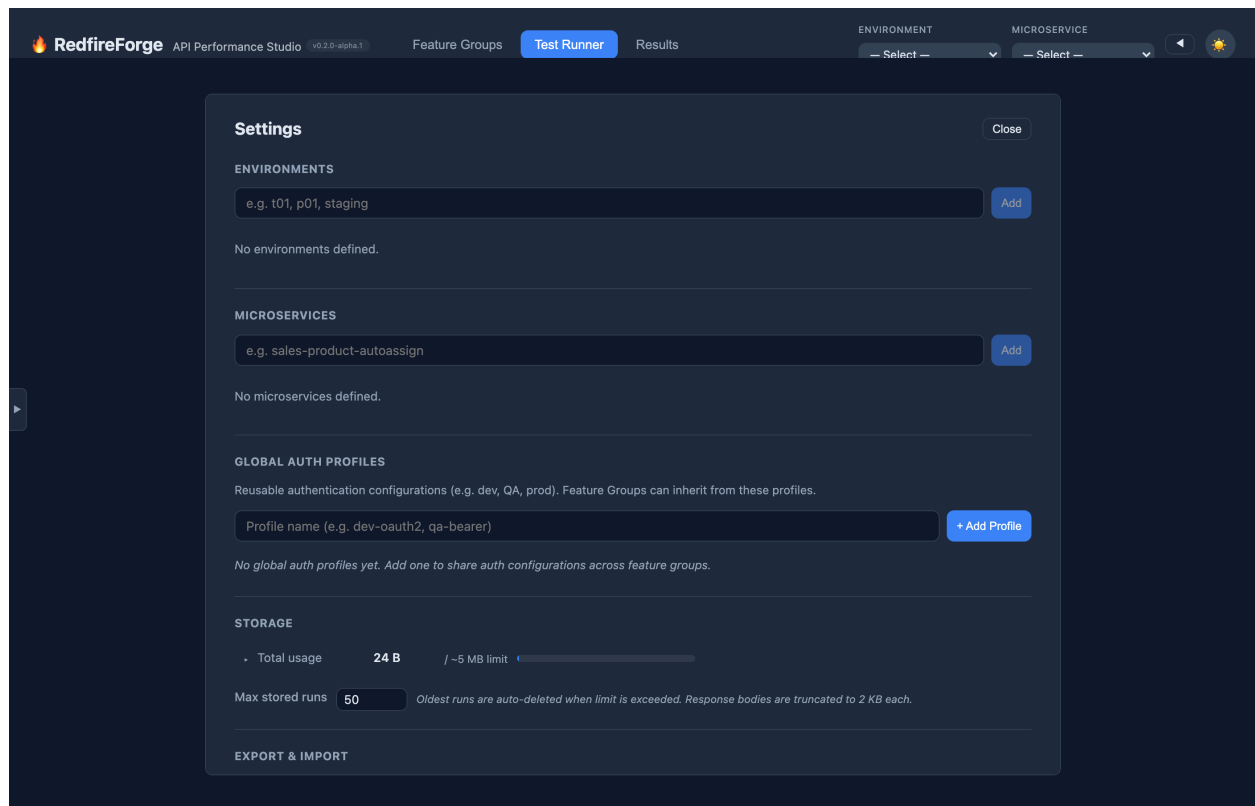


Figure 7: Test Runner — configuration and execution

Execution Modes

Mode	Behavior	Best For
Sequential	One request at a time	Baseline measurements
Parallel	All requests at once	Load testing
Ramp-up	Gradually increase concurrency	Stress testing

Step 1: Select What to Test

Choose a Feature Group, Scenario, or individual Test from the dropdown.

Step 2: Configure Iterations

Set the number of runs (iterations) to execute.

Step 3: Choose Execution Mode

Select Sequential, Parallel, or Ramp-up mode.

Step 4: Click Run

Press the Run button. Watch real-time progress as tests execute. Results are automatically saved when complete.

10. Analyzing Results

The Results tab shows all historical test runs with detailed metrics.

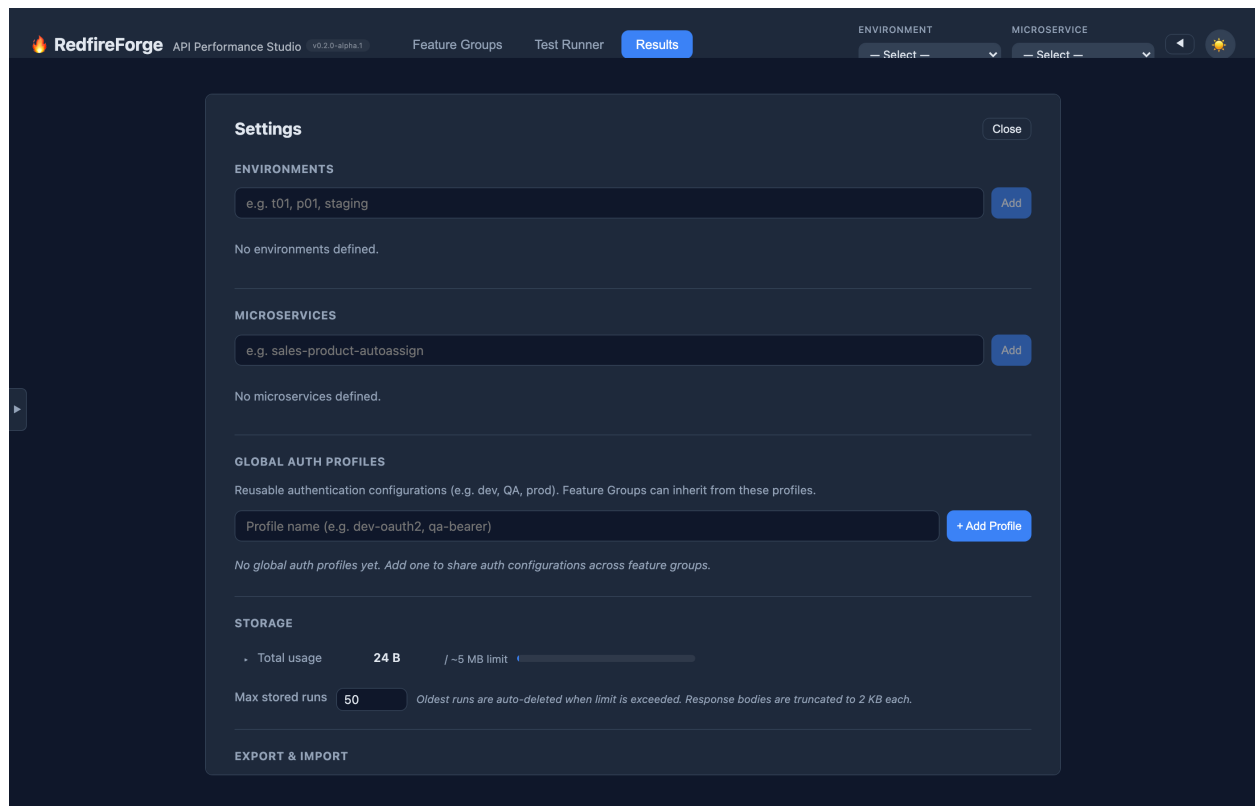


Figure 8: Results Dashboard — historical test runs

Available Metrics

- **Average Response Time** — mean of all request durations
- **Min / Max** — fastest and slowest responses
- **P95 / P99** — 95th and 99th percentile response times
- **Error Rate** — percentage of failed requests
- **Status Code Distribution** — breakdown by HTTP status

You can export results as JSON or CSV for further analysis in spreadsheets.

11. Export & Import

Export Center

Access via Settings → Export Center. Export your entire configuration or select specific items. Files are saved to **Documents/RedfireForge/** by default.

Exported files use a standardized naming convention:

`{environment}-{microservice}-{level}-{name}-{timestamp}.json`

Import Center

Access via Settings → Import Center. Select a previously exported .json file. RedfireForge will detect conflicts and let you resolve them per item:

Action	Behavior
Skip	Keep existing, ignore imported item
Overwrite	Replace existing with imported item
Keep Both	Import as a new item with a new ID

12. Desktop vs Web Comparison

Feature	Desktop (Tauri)	Web (Browser)
Storage	OS AppData directory (files)	Browser localStorage
HTTP Requests	Native HTTP (no CORS issues)	Vite proxy required
File Dialogs	Native OS file dialogs	Browser file picker
Default Export Dir	Documents/RedfireForge/	Browser default
Installation	.dmg / .msi / .AppImage	npm run dev
Offline Use	Yes — fully standalone	No — needs dev server
Data Persistence	Survives browser cache clears	Lost if cache cleared

The desktop application is recommended for daily use. The web version is useful for quick UI development and testing.

13. Tips & Best Practices

Organize by Environment

Create one Feature Group per environment/microservice combination for clarity.

Use Global Auth Profiles

Define credentials once in Settings → Global Auth Profiles. Let tests inherit rather than duplicating auth config in every test.

Export Regularly

Export your configuration as a backup. Use Documents/RedfireForge/ to keep all exports organized.

Start Sequential, Then Parallel

Validate your tests in Sequential mode first. Once they pass, switch to Parallel or Ramp-up for load testing.

Monitor P95/P99

Average response time can hide outliers. Always check P95 and P99 for a realistic picture of user experience.

Use Drag-and-Drop

As your test suite grows, use drag-and-drop to restructure scenarios and tests across Feature Groups.

Prefer Desktop App

The desktop app has no CORS issues, uses native file dialogs, and stores data independently of browser cache.

14. Quick Reference

Action	Where	How
Switch views	Header bar	Click Feature Groups / Test Runner / Results
Filter by env	Sidebar	Click an Environment
Filter by service	Sidebar	Click Microservices tab
Create Feature Group	Feature Groups tab	Click '+ Add Feature Group'
Add Scenario	Inside Feature Group	Click '+ Add Scenario'
Add Test	Inside Scenario	Click '+ Add Test'
Reorder items	Feature Groups	Drag and drop
Settings	Sidebar	Click  Settings
Export config	Settings modal	Click Export Center
Import config	Settings modal	Click Import Center
Run tests	Test Runner tab	Configure → Click Run
View results	Results tab	Browse historical runs

■ RedfireForge

Fire. Measure. Validate.

Version 0.2.0